

一、选择题

1.

要实现对象 a 具有 a++ 操作符的功能，以下写法正确的是：
()

- A. a &operator ++();
- B. a &operator ++(int);
- C. a &operator ()++;
- D. a &operator (int)++;

答案：B

分析：C++ 后置加加 重载的固定格式—>class_name operator ++(int);

2.

2. 下列说法正确的是：()

对于程序：

int a[10];

int b[2][5];

- A. a 的内存空间是在堆上申请的。
- B. a 所占用的内存长度是 10 个字节。
- C. b 所占用的内存比 a 多。
- D. b 所申请的内存每一维都是连续的。
- E. a 和 b 默认的元素都是 0 值。

答案：D

分析：

- A - 堆上内存是用 calloc 或 malloc 申请的，所以 a 的内存不是在堆上
- B - a 是整型数组，它的每个元素都是整型，因此至少占 10*2 个字节(16 位系统)
- C - b 和 a 的数组元素个数一样，所以占内存也是一样的
- E - a 和 b 都没有初始化，所以他们的值都是随机值

3.

3. 以下说法正确的是：()

- A. 红黑树的左右两颗子树的高度差的绝对值，不超过 2。
- B. 哈希表的搜索速度不但取决于哈希算法的好坏，还取决于桶的数量。
- C. 堆栈是一种先进先出的数据结构。
- D. 在一个已排序链表中使用二分搜索法搜索数据是一个快速的方法。

答案：A

分析：

- B - 哈希表的搜索速度只取决于哈希算法
- C - 栈是先进后出
- D - 哈希表、二叉树都有可能比二分法快，而且二分法更适合数组查询

4.

4. 以下哪种方法分配内存的速度最快? ()

- A. 在堆栈上分配内存。
- B. 在堆上分配内存。
- C. 使用内存池算法分配内存。

答案: A

分析: B 和 C 获得的内存都是堆上的内存, 而 A 获得的内存是栈上的, 栈上的内存分配永远比堆上的快

知识点: 内存池 (Memory Pool) 是一种内存分配方式。通常我们习惯直接使用 new、malloc 等 API 申请分配内存, 这样做的缺点在于: 由于所申请内存块的大小不定, 当频繁使用时会造成大量的内存碎片并进而降低性能。内存池则是在真正使用内存之前, 先申请分配一定数量的、大小相等 (一般情况下) 的内存块留作备用。当有新的内存需求时, 就从内存池中分出一部分内存块, 若内存块不够再继续申请新的内存。这样做的一个显著优点是, 使得内存分配效率得到提升

5.

5. 以下关于内存的使用, 哪些说法是正确的 ()

- A. 以 8 字节对齐的结构定义一定比以 16 字节对齐的结构定义适用内存省。
- B. 内存频繁分配和释放, 在长时间后容易造成内存碎片过多, 无法分配内存。
- C. 类对象中, 成员函数只在整个程序中占用一份内存, 成员变量占用的内存数量同对象数量有关。
- D. 定位 new 操作分配的对象, 不会自动调用析构函数。

答案: B

分析:

A 选项表达的是 8 字节对齐比 16 字节对齐省内存, 但是 8 个字节和 16 个字节有的时候没有区别。若使用了 #pragma pack(n) 命令强制对齐标准, 则取 n 和结构体中最长数据类型占的字节数两者之中的小者作为对齐标准。

```
test1.c (-jxc) - VIM 36x52 | test1.c (-jxc) - VIM 37x52 | iotek@zjx: yxc$ ./pack8
1 #include <stdio.h> | 1 #include <stdio.h> | 44
2 #pragma pack(16) | 2 #pragma pack(8) | iotek@zjx: yxc$ ./pack16
3 | 3 | 44
4 typedef struct user{ | 4 typedef struct user{ | iotek@zjx: yxc$
5     long long time; | 5     long long time; |
6     int type; | 6     int type; |
7     char name[20]; | 7     char name[20]; |
8     char choose; | 8     char choose; |
9     double score; | 9     double score; |
10 }User_t; | 10 }User_t; |
11 | 11 |
12 int main(void) | 12 int main(void) |
13 | 13 |
14     printf("%d\n", sizeof(User_t)); | 14     printf("%d\n", sizeof(User_t)); |
15     return 0; | 15     return 0; |
16 | 16 |
```

C 单例模式的内存数量和对象数量没有关系

D 析构函数都是自动调用的

知识点: 作者: 刘冠宇

链接: <https://www.zhihu.com/question/51836333/answer/144508812>

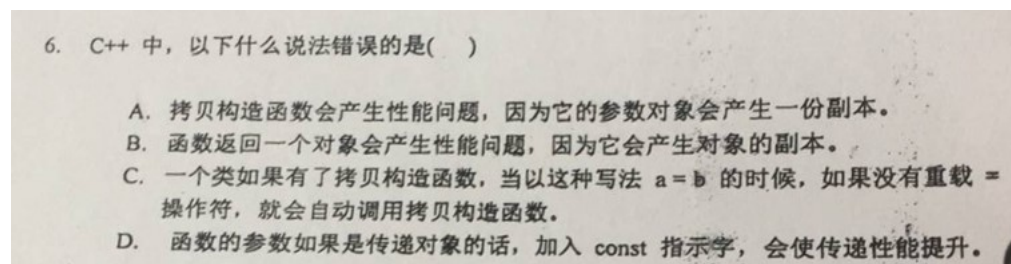
来源: 知乎

著作权归作者所有。商业转载请联系作者获得授权, 非商业转载请注明出处。

现代计算机中内存管理使用了基于虚拟内存的分段和分页机制, 每一个进程看到的

是 4G（32 位）或 64TB（64 位）的虚拟内存地址空间，一般情况程序是用不完的。分页机制保证仅当程序需要储存修改数据的时候才会在真实的物理内存上获取多个 4KB 大小的不连续的页，且被初始化为全零，这些页作为虚拟内存上的映射，使用一个 malloc，操作系统会在进程的虚拟内存分配表上标记那块区域已被使用，程序修改数据的时候才会在物理内存中开辟空间获取多个不连续的页，使用 free，操作系统会删除内存分配表的记录，同时回收物理内存上的页。使用 malloc 和 free 虽然会导致虚拟内存上碎片，但在物理内存上是不连续使用的，不会导致碎片。在普通计算机上如果虚拟内存碎片太多无法分配，一般操作系统会告诉你进程无响应，重新打开程序即可，一般 64TB 空间不会出现这种情况；长时间运行的操作系统可能会有这种情况，此时重启即可。在服务器上，或持久运行的操作系统如核电站上，编译器一般采用句柄池的方法，即二次指针，以 64 位机为例，malloc 获取指向句柄池的一个元素的指针 p，p 指向指针 p1，p1 才指向 malloc 获取的地址，当内存碎片到一定程度时，操作系统会将内存中对象向一段移动从而压缩碎片，这时只需要修改句柄池中的 p1 即可；但这样效率太低，只有追求高可靠运行的系统才会使用。碎片只会在虚拟内存中产生，是不会映射到物理内存上的。

6.



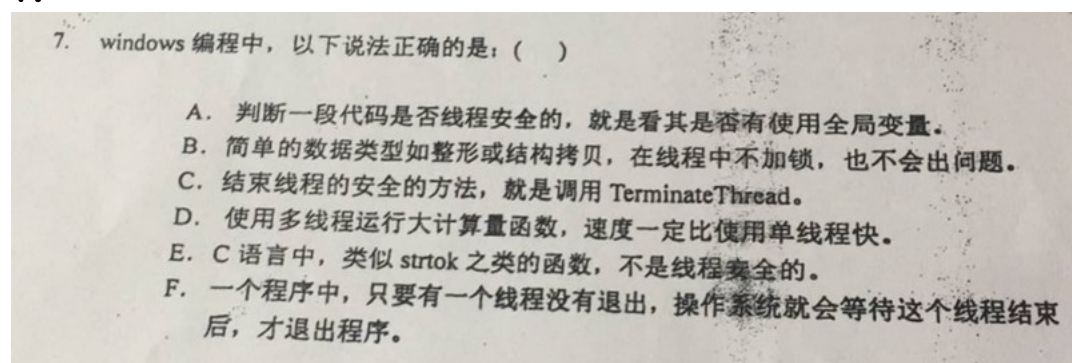
答案：D

分析：const 关键字只是修饰变量为只读，防止程序或者编译器修改

知识点：拷贝构造函数的调用时机

1. 对象以值传递的方式传入函数参数
2. 对象以值传递的方式从函数返回
3. 对象需要通过另外一个对象进行初始化

7.



答案：E

分析：

A 如果对全局变量上锁，也是可以保证线程安全的

B 是否上锁要看类型或结构是否是全局的

C 线程结束的最安全办法是线程函数返回，`TerminateThread` 是异步运行的函数，也就是说，它告诉系统你想要线程终止运行，但是，该函数返回时，不能保证线程

被撤销。

D 运行单个函数，多线程和单线程没区别

E strtok 函数会修改字符数组的内容，所以在完整分割完之前被分割的数组不能被其他线程访问，所以 strtok 函数是线程不安全的

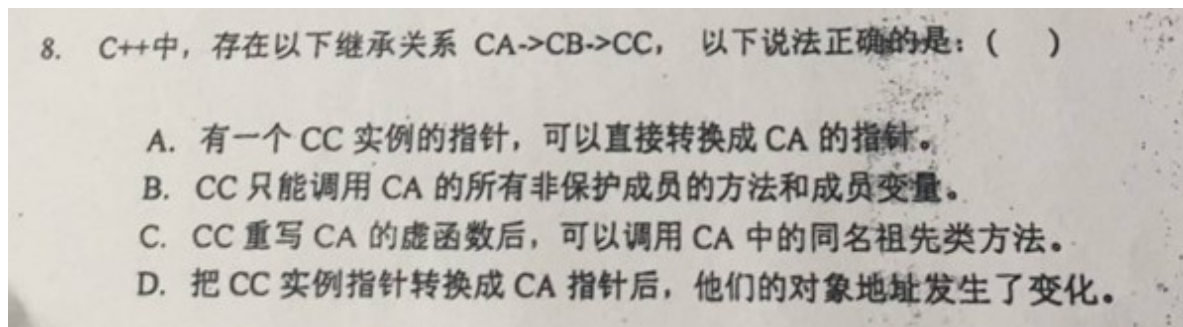
F 进程退出了所有线程都会被强制退出

知识点：windows 操作系统中，创建的线程有以下 4 种终止方式

1. 线程函数返回
2. ExitThread 函数
3. TerminateThread 函数
4. 进程终止运行时终止线程

通常，应当始终采用第一种方法来终止线程的运行。

8.



答案：A

分析：

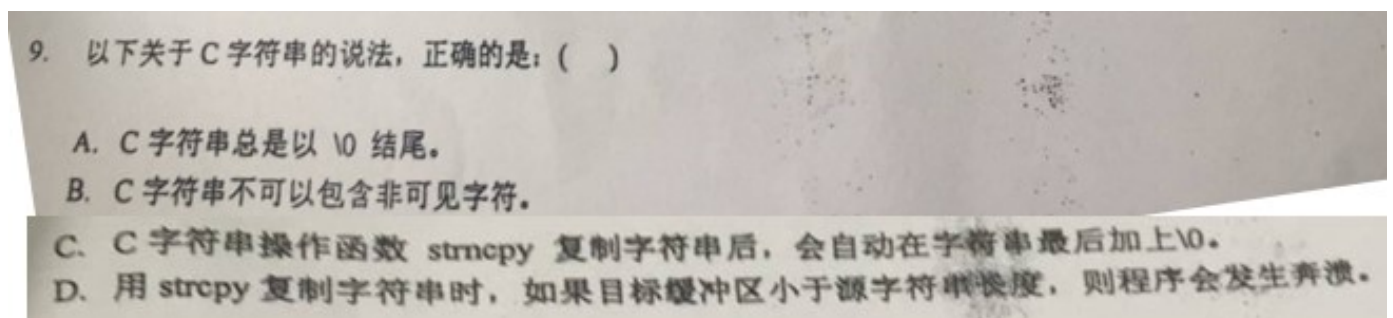
A 派生类对象是基类对象，派生类中包含有基类的成员。基类对象不是派生类对象，它不能包含派生类型的成员。

B 不能调用私有成员，私有成员也是非保护成员

C 不能用基类指针给派生类指针赋值，而且多态是基类访问派生类

D 对象的地址不会发生改变，类型转换但是地址值不会变换

9.



答案：A，C

分析：

A C 字符串的定义：字符串就是一串以 \0 字节结尾的字符序列

B 可以包含非可见字符如 \0

C strncpy 系统源函数中的确会字符串后面加上 \0

D 程序是否崩溃要看目标缓冲区后面的内存是否被占用

10.

10. windows 编程中，当你在编写一个子窗口控件，你想让父窗口响应到你所产生的事件，最正确的做法是：()

- A. 向父窗口发送一条自定义消息。
- B. 向父窗口发送一条通知消息。
- C. 在创建子窗口的时候，先把父窗口指针设置给其，然后在想产生事件的时候通过指针调用父窗口的方法。
- D. 通过 GetParent 方法获得父窗口的句柄，再通过 MFC 函数 FromHandle 获取到 CWnd 指针，然后转换成父窗口类指针调用其方法。

答案：B

分析：

父子窗口通信系统有默认的消息机制 WM_COMMAND，子窗口控件发送通知消息给父窗口就可以唤醒对应的响应了

因为子窗口控件是用系统窗口类创建的，所以内部的消息处理都是系统定制好了的。所以无法给你提供个性化的消息，发不了自定义消息。

而父窗口要告诉子窗口控件怎么处理现在的情况，则需要子窗口控件的窗口句柄。所以在 LPARAM 参数中，已经携带了子窗口控件的窗口句柄。这个句柄是子窗口控件让父窗口好给自己发消息，因为发消息要用窗口句柄。

11.

11. windows 编程中，以下说法错误的是：()

- A. 所有的 windows 消息响应函数，都有一个返回值。
- B. 消息机制是不可靠的，发送的消息，可能会丢失。
- C. 一个应用程序的窗口，不可以跨进程发送给另一个应用程序窗口消息。
- D. 消息的处理顺序为：应用程序消息循环->窗口消息过程->具体的消息处理函数。

答案：C

分析：

A Win32 消息处理后必须返回 0，窗口过程中必须有默认消息处理函数

B Windows 消息机制是和 Windows 系统挂钩的，所有的消息都放在消息队列当中，计算机内存不足会使队列存满，导致信息丢失

C 两个 console 间完全可以通过消息进行通讯，不过不通过窗口句柄的话只有 PostThreadMessage() 可用。

D 应用程序必须对投递在他的线程队列中的消息进行取出和处理。单线程的应用程序通常在 WinMain 函数使用一个消息循环来取出消息，发送消息给对应的窗口过程函数来对消息进行处理。多线程的应用程序在每个创建窗口的线程中都可以包含一个消息循环。

12. 以下控件中，() 没有 Caption 属性

A: 按钮 B: 群组框 C: 编辑控件 D: 静态控件

答案：B

分析：窗体的属性主要可分为：杂项、外观、行为、字体、位置等五类。

Caption: 窗体标题，是出现在窗体标题栏上的文字内容，是给使用者看的；

Name: 窗体名称, 在程序代码中被作为对象的标识名, 是给程序用的。

具有 Caption 属性而没有 Text 属性的控件有: 标签、命令按钮、复选框、选项按钮、框架;

具有 Text 属性而没有 Caption 属性的控件有: 文本框、组合框、列表框。

群组框又称为组合框

13. 更新菜单状态的消息是()

A: WM_COMMAND

B: UPDATE

C: ON_UPDATE_COMMAND_UI

D: INVALIDATE

答案: C

分析: MFC 提供了一种更新菜单项, 通过消息映射表中的 ON_UPDATE_COMMAND_UI 宏, 可以给单个菜单项分配选中的成员函数作为更新处理程序。当用户点一个下拉菜单时, MFC 捕获随之产生的 WM_INITMENUPOPUP 消息, 并调用菜单中所有菜单项的更新处理程序。

14. 关于工具栏、菜单和加速键的关系, 正确的说法是()

A: 工具按钮与菜单必须一一对应

B: 工具按钮与加速键一一对应

C: 工具按钮不必与菜单一一对应

D: 菜单与加速键一一对应

答案: C

分析:

A: 不是每个菜单都需要按钮

B: 加速键和菜单有关

D: 加速键可以在不同菜单中对应不同的选项

15. 使用 GetWindowDC() 和 GetDC() 获取的设备上下文在退出时, 必须调用() 释放设备上下文。

A: ReleaseDC()

B: delete()

C: DeleteDC()

D: Detach()

答案: A

分析: 固定的知识点, GetDC 和 ReleaseDC 一一对应

16. 在多进程的并发系统中, 肯定不会因竞争() 而产生死锁

A: 硬盘

B: 内存

C: CPU

D: 打印机

答案: C

分析:

CPU 属于可剥夺性资源, 但一个进程已获得的资源, 在未使用完之前, 另一个进程要使用可以把它资源剥夺过来, 所以不会产生死锁。只有不可剥夺资源才会因为竞争资源而产生死锁。

硬盘、内存、打印机都是共享资源, 有可能被一个进程独占

17. 本地用户通过键盘登录系统时，首先获得键盘输入信息的程序是()

- A: 命令解释程序 B: 系统调用程序
C: 中断处理程序 D: 用户登录程序

答案: C

分析：当用户按键时，键盘接口会得到一个代表该按键的键盘扫描码，同时产生一个中断请求。键盘中断服务程序先从键盘接口取得按键的扫描码，然后根据其扫描码判断用户所按的键并作相应的处理，最后通知中断控制器本次中断结束并实现中断返回。因此，本地用户通过键盘登录系统时，首先获得键盘输入信息的程序是中断处理程序。

18. 在 Linux 系统中，使用 `ls -a` 命令，输出显示的 “.” 想表示()

- A: 临时文件 B: 隐藏文件夹 C: 当前目录 D: 以上都不是

答案: C

分析：固定的知识点，一个点是当前目录，两个点是上一级目录，类比 `cd ./` 和 `cd ../`

19. IPv6 将 32 位地址扩展到()

- A: 64 位 B: 128 位 C: 256 位 D: 1024 位

答案: B

分析：固定的知识点，IPv6 的地址长度为 128b，是 IPv4 地址长度的 4 倍。于是 IPv4 点分十进制格式不再适用，采用十六进制表示

20. 电子邮件应用程序利用 POP3 协议()邮件

- A: 创建 B: 加密 C: 发送 D: 接收

答案: C

分析：POP3 是 Post Office Protocol - Version 3 的缩写，即邮局协议版本 3，是一种用来从邮件服务器上读取邮件的协议，用于将邮件从 POP 服务器传送到用户代理。负责接收邮件的用户代理称为 POP 客户，负责发送邮件的服务器称为 POP 服务器。

二、阅读程序

1、

```
void GetMemory(char *p)
{
    p = (char*)malloc(100);
}
void Test(void)
{
```

```

    char *str = NULL;
    GetMemory(str);
    strcpy(str, "hello world");
    printf(str);
}

```

请问运行 Test 函数会有什么样的结果？

答案：会出现段错误，虽然调用 GetMemory 函数传入了指针 str，但是 GetMemory 函数的形参 p 替换了实参 str，因此 malloc 出来的内存和 str 无关，那么给空指针 str copy 字符串则一定会出现段错误

知识点：段错误出现的常见情况：

- 1、使用非法指针，即野指针
- 2、对 NULL 指针访问；
- 3、修改常量区中的内容
- 4、数组越界（并非数组越界一定会报错，与系统环境有关）
- 5、多次释放同一块内存
- 6、跨进程访问某个地址，即访问非法区域

2、

```

void GetMemory(void)
{
    char p[] = "hello world";
    return p;
}

void Test(void)
{
    char *str = NULL;
    str = GetMemory();
    printf(str);
}

```

请问运行 Test 函数会有什么样的结果？

答案：会打印 NULL，虽然调用 GetMemory 函数，p 指针也能返回，但是 p 指向的字符串是存放在栈空间，函数结束后栈空间释放，p 指向的空间就是 NULL（有的系统是默认为野指针），因此 str 也是指向了 NULL 区域，所以打印效果为 NULL

3、

```

void Test(void)
{
    char *str = (char*)malloc(100);
    strcpy(str, "hello");
    free(str);
}

```



```

        if(str != NULL)
        {
            strcpy(str, " world" );
            printf(str);
        }
    }
}

```

请问运行 Test 函数会有什么样的结果？

答案：会出现段错误，当 str 指向的堆空间被 free 后 str 已经变成了野指针，str 的值不为 NULL，因此给 str copy “world” 就变成了操作野指针了

4、

```

void Test(void)
{
    int i = 1;
    int j = i++;
    if((i > j++) && (i++ == j)) i+=j;
    printf( "%d" ,i);
}

```

请问运行 Test 函数会有什么样的结果？

答案：5

分析：考察后加加的应用，运算优先级，结合性

初始化时 i=1，而 j=i++，这时 i 是先赋值在加加，因此 j=1，i=2；

i > j++，j 是先参与逻辑运算再加加，因此 i>j 为真，i=2，j=2；

i++ == j，i 是先参与逻辑运算再加加，因此 i == j 为真，i=3，j=2；

关系&&两边都是真，因此执行 i+=j；i = i + j；i=5；

5、

```

char ch[5] = "lisi" ;
char * const pStr = ch;
pStr = "wangwu" ;
cout << pStr << endl;

```

请问运行以上的代码会有什么样的结果？

答案：编译无法通过，pStr = “wangwu” 这一行编译会报错，pStr 是 const 修饰的只读指针，只能初始化，无法再重新赋值(修改)

6、

```

char ch[5] = "Lisi" ;
char * const pStr = ch;
*pStr = 'W' ;

```

```
cout << pStr << endl;
```

请问运行以上的代码会有什么样的结果？

答案： 会输出 Wisi

分析：pStr 的值不可修改，但是 pStr 指向的内容可以修改，因此*pStr 就是数组 ch 的首元素，因此 L 被修改成了 W，所以输出 Wisi

三、简答题

1、在 C++ 程序中调用被 C 编译器编译后的函数，为什么要加 extern C 声明？

答案：C++ 语言支持函数重载，C 语言不支持函数重载。函数被 C++ 编译后在库中的名字与 C 语言的不同。假设某个函数的原型为：void foo(int x, int y); 该函数被 C 编译器编译后在库中的名字为 _foo，而 C++ 编译器则会产生像 _foo_int_int 之类的名字。C++ 提供了 C 连接交换指定符号 extern “C” 来解决名字匹配问题

2、

存在一个“人”的 C++ 类，这个类派生出 2 个子类，一个是“天使”，一个是“恶魔”。现在要为这两个派生类，分别加上“会飞”这个方法的实现，应该怎么办？
(8 分)

答案：多态的实现

- 1. 在基类人中添加“会飞”的虚函数**
- 2. 在两个派生类中重写“会飞”的函数，用不同的方法去实现**
- 3. 用基类的指针就可以调用不同派生类对象了**

知识点：C++ 的多态：

多态成立的三个条件

- 1 要有继承
- 2 要有函数重写——虚函数
- 3 要有父类指针（父类引用）指向子类对象

虚函数

1 构造函数不能是虚函数。建立一个派生类对象时，必须从类层次的根开始，沿着继承路径逐个调用基类的构造函数

2 析构函数可以是虚的。虚析构函数用于指引 delete 运算符正确析构动态对象

理解四个名词：

- 1、对象的静态类型：对象在声明时采用的类型。是在编译期确定的。
- 2、对象的动态类型：目前所指对象的类型。是在运行期决定的。对象的动态类型可以更改，但是静态类型无法更改。
- 3、静态绑定：绑定的是对象的静态类型，某特性（比如函数）依赖于对象的静态类型，发生在编译期。
- 4、动态绑定：绑定的是对象的动态类型，某特性（比如函数）依赖于对象的动态类型，发生在运行期。

以 virtual 函数为例。当某个 virtual 函数通过指针或引用调用时，编译器产生的

代码直到运行时才能确定应该调用哪个版本的函数。被调用的函数是与绑定到指针或引用上的对象的动态类型相匹配的那一个。

3、

小明使用递归算法写了个遍历树结构的方法，但是由于树的层次太深，递归算法发生了“堆栈溢出”错误，请问如何解决？(8分)

答案：1. 明确递归退出的界限，及早的退出递归循环；2. 根据硬件的条件给递归算法添加递归次数的限定，例如递归调用不能超过 1000 次；3. 使用 static 对象替代 nonstatic 局部对象，在递归函数设计中，可以使用 static 对象替代 nonstatic 局部对象（即栈对象），这不仅可以减少每次递归调用和返回时产生和释放 nonstatic 对象的开销，而且 static 对象还可以保存递归调用的中间状态，并且可为各个调用层所访问。

知识点：什么是栈溢出？

因为栈一般默认为 1-2MB，一旦出现死循环或者是大量的递归调用，在不断的压栈过程中，造成栈容量超过 1MB 而导致溢出。

4、

3. 小李在写程序的时候，发现需要访问 ClassA 类的私有成员变量，于是他就修改了 ClassA 类，加入了 GetXXX, 和 SetXXX 两个方法，分别用来返回和设置这个成员变量，他的做法对吗，为什么？(10分)

答案：下面列举访问“类私有成员变量”的 3 种方法：

1. 通过类公有成员函数 set(), get() 写、读私有成员；
2. 通过类公有成员函数 get() 获取私有成员的指针或引用，对指针或引用操作；
3. 通过友元。

方法 1, 3 都是经常使用的，方法 2 有点意外；不管怎么说，方法 2, 3 都破坏了类的封装性。

5、如果你有无穷多的水，一个 3L 的和一个 5L 的杯子，你如何准确称出 4L 的水？

答案：加满 3L, 全部倒到 5L, 再加满 3L, 用 3L 把 5L 装满, 这样 3L 中就剩下 1L, 把 5L 的水倒掉, 把 3L 剩下的 1L 倒到 5L 中, 然后再加满 3L, 倒到 5L 中, 就拥有 4L 的水了。

6、桌子上原来有 12 支点燃的蜡烛，先被风吹灭了 3 根，不久又一阵风吹灭了 2 根，最后桌子上还剩几根蜡烛？

答案：分两种情况

1. 蜡烛全部燃烧结束，桌子上还剩 5 根
2. 蜡烛没有燃烧到结束，桌子上依旧有 12 根蜡烛

7、你让工人为你工作 7 天，给工人的汇报是一根金条，金条平分成相连的 7 段，你必须在每天结束时给他们一段金条，如果只允许你两次把金条弄断，你如何给工人付费？

答案：两刀把金条切成三段，分成 $1/7$ 、 $2/7$ 、 $4/7$ 。

第一天 $1/7$ ，工人手中有一段金条；

第二天给 $2/7$ ，找回 $1/7$ ，工人手中有两段金条；

第三天再给 $1/7$ ，工人手中有三段金条；

第四天给 $4/7$ ，找回 $1/7$ 和 $2/7$ ，工人手中有四段金条；

第五天给 $1/7$ ，工人手中有五段金条；

第六天给 $2/7$ ，换回 $1/7$ ，工人手中有六段金条

第七天把最后的一份 $1/7$ 的给工人，工人手中有七段金条

8、一个人花 8.8 元钱买了一只鸡，9.8 元钱卖掉了，然后他觉得不划算，花了 10.8 元钱又买了回来，11.8 元钱卖给了另外一个人，问他是亏了还是赚了？

答案：

1. 赚了 1 元

整个事件有 3 次交易

第一次交易：8.8 元买进，9.8 元卖出，利润 1 元；

第二次交易：9.8 元卖出，10.8 元买进，利润-1 元；

第三次交易：10.8 元买进，11.8 元卖出，利润 1 元；

所以赚了 1 元

2. 赚了 2 元

$-8.8+9.8-10.8+11.8 = 2$

3. 赚了 3 元

最初只有 8.8 块钱，最后你有 11.8 块了，所以是赚 3 块；

4. 亏了 2 元

本来可以直接赚 3 元的，经过他 3 次交易后总利润变成 1 元了，所以是亏了 2 元

5. 亏了 3 元

两次买的交易，两次卖的交易，如果每次买入一只鸡都是花了 8.8 元，每次卖出一只鸡都是 11.8 元，所以最高利润可以是 6 元，如果是以赚 3 元的角度算亏了 3 元

6. 亏了 4 元

最高利润是 6 元，如果是以赚 2 元的角度，所以是亏了 4 元

7. 亏了 5 元

最高利润是 6 元，如果是以赚 1 元的角度，所以是亏了 5 元

9、

4. 当你处于一个团队中进行开发，当另一个程序提供给你的 dll 调用有问题（没有源代码），而且程序有时候运行结果是正确的，少数时间是错误的，你怀疑问题出在他的程序中，你会怎么处理？（8 分）

答案:

- 1) 找出错误的情况, 看看能不能把这种情况在自己程序限定住, 即函数调用时, 给传递的参数限定条件, 避免可能导致错误的参数传递过去; 例如 d11 有个中间变量它是 int 型, 导致大数据计算时无法保留正确的计算结果, 因此程序 bug; 有或者数据类型不匹配等情况
- 2) 看看是否有版本限定的问题, 32 位的系统和 64 位的系统在指针操作方面有细微的差别, ios 版本和 Windows 版本的差别, C++编写的程序与 C 编写的程序也有差别, C98 和 C11 的差别等
- 3) d11 中可能有不合理的系统调用, 当涉及到系统调用的时候, 因系统管理权限不够无法运行
- 4) 也有可能对方的 d11 算法有问题, 忽略了一些特殊情况的出现, 如果项目要求是不能忽略这些特殊情况, 则需要找 d11 提供者请他把算法再优化一下